## WEEK-2

### SESSION-6:

### TASK-1:Shuffle The Array

```
class Solution {
        public int[] shuffle(int[] nums, int n) {
                int[] result = new int[2 * n];
                for (int i = 0; i < n; i++) {
                        result[2 * i] = nums[i];
                        result[2 * i + 1] = nums[n + i];
    }
    return result;
  }
}
```

### SESSION-7:

### TASK-1:REMOVE ELEMENT

```
class Solution {
  public int removeElement(int[] nums, int val) {
    int k = 0;
    for (int i = 0; i < nums.length; i++) {
      if (nums[i] != val) {
        nums[k] = nums[i];
        k++;
      }
    }
```

```
        return k;
    }
}
```

**TASK-2:REMOVE DUPLICATES FROM SORTED ARRAY**

```
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int k = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[i - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }
        return k;
    }
}
```

**SESSION-8:**

**TASK-1:MAXIMUM SUBARRAY**

```
class Solution {
    public int maxSubArray(int[] nums) {
        int target=nums[0];
        HashSet<Integer> s=new HashSet<>();
        for(int x:nums){
```

```
            if(set.contains(target-x)){
                break;
            }
            set.add(x);
        }
        return target;
    }
}
```

**TASK-2:FIND THE HIGHEST ALTITUDE**

```
class Solution {
    public int largestAltitude(int[] gain) {
        int currentAltitude = 0;
        int maxAltitude = 0;

        for (int g : gain) {
            currentAltitude += g;
            maxAltitude = Math.max(maxAltitude, currentAltitude);
        }

        return maxAltitude;
    }
}
```

**SESSION-9:**

**TASK-1:GROUP ANAGRAMS**

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        if (strs == null || strs.length == 0) {
            return new ArrayList<>();}
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] charArray = s.toCharArray();
            Arrays.sort(charArray);
            String sortedKey = new String(charArray);
            if (!map.containsKey(sortedKey)) {
                map.put(sortedKey, new ArrayList<>());
            }
            map.get(sortedKey).add(s);
        }
        return new ArrayList<>(map.values());
    }
    }
```

**TASK-2:TOP K FREQUENT ELEMENTS**

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> frequencyMap = new HashMap<>();
        for (int num : nums) {
```

```
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0)
+ 1);
        }
        PriorityQueue<Integer> minHeap = new PriorityQueue<>(
            (a, b) -> frequencyMap.get(a) - frequencyMap.get(b)
        );
        for (int num : frequencyMap.keySet()) {
            minHeap.add(num);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
            }
        int[] result = new int[k];
        for (int i = k - 1; i >= 0; i--) {
            result[i] = minHeap.poll();
        }

        return result;
    }
}
```

**SESSION-10:**

**TASK-1:JAVA DEQUEUE**

```
import java.util.*;
public class test {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        Deque<Integer> deque = new ArrayDeque<>();
        HashMap<Integer, Integer> map = new HashMap<>();

        int n = in.nextInt();
        int m = in.nextInt();
        int maxUnique = 0;

        for (int i = 0; i < n; i++) {
            int num = in.nextInt();
            deque.addLast(num);
            Integer count = map.get(num);
            if (count == null) {
                map.put(num, 1);
            } else {
                map.put(num, count + 1);
            }

            if (deque.size() > m) {
```

```
            int removed = deque.removeFirst();
            map.put(removed, map.get(removed) - 1);
            if (map.get(removed) == 0) {
                map.remove(removed);
            }
        }

        if (deque.size() == m) {
            maxUnique = Math.max(maxUnique, map.size());
        }
    }
    System.out.println(maxUnique);
    in.close();
  }
}
```

**TASK-2:JAVA HASHSET**

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
```

```
 public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int t = s.nextInt();
        String [] pair_left = new String[t];
        String [] pair_right = new String[t];

        for (int i = 0; i < t; i++) {
            pair_left[i] = s.next();
            pair_right[i] = s.next();
        }
 HashSet<String> set = new HashSet<String>();
        for (int i = 0; i < t; i++) {
            set.add(pair_left[i] + " " + pair_right[i]);
            System.out.println(set.size());
        }

    }
}
```